UTC

REPORTE TECNICO DE MACHINE LEARNING

Implementacion de Modelos KNN y Arbol de Decision

ALUMNO:	Sergio Alejandro Riancho Meza
MATRICULA:	240132402
GRUPO:	6to Cuatrimestre
CARRERA:	Ingenieria en Sistemas Computacionales
ASIGNATURA:	Inteligencia Artificial
DOCENTE:	Mtro. Gonzalo Ivan Riego Caravantes
FECHA:	24 de Marzo de 2026

REPORTE TECNICO: CLASIFICACION SUPERVISADA (KNN & ARBOL DE DECISION)

UNIVERSIDAD TRES CULTURAS

Ingenieria en

1. Portada

* Universidad: Universidad Tres Culturas (UTC)
* Carrera: Ingenieria en Sistemas Computacionales
* Asignatura: Inteligencia Artificial / Machine Learning
* Titulo: Examen Practico Integrador por Etapas - KNN y Arbol de Decision con Simulador Web
* Alumno: Sergio Alejandro Riancho Meza
* Matricula: 240132402
* Grupo: 6to cuatrimestre
* Docente: Mtro. en C.C. Gonzalo Ivan Riego Caravantes
* Fecha: 24 de Marzo de 2026

2. Introduccion

La clasificacion supervisada es una de las tareas fundamentales del aprendizaje automatico (Machine Learning). Su utilidad radica en la capacidad de entrenar sistemas que aprendan de datos historicos etiquetados para despues tomar decisiones automaticas sobre nuevos datos. Esto tiene aplicaciones directas en medicina (diagnostico), calidad industrial, finanzas y ciencias de la computacion.

En este proyecto se implementan dos clasificadores supervisados clasicos: K-Nearest Neighbors (KNN) y Arbol de Decision, y se integran en un simulador web interactivo. El sistema permite al usuario cargar datasets reales, configurar parametros, entrenar modelos, visualizar metricas de desempeno y realizar predicciones tanto individuales como masivas, demostrando en forma completa el ciclo de vida de un modelo de Machine Learning.

3. Objetivo

Desarrollar un sistema practico de clasificacion supervisada que combine programacion en Python y

desarrollo web, cumpliendo con todos los requerimientos del examen integrador de la UTC:

1. Implementar KNN y Arbol de Decision como programas independientes en Python.
2. Integrar ambos algoritmos en un simulador web funcional con backend Flask.
3. Utilizar los datasets obligatorios: Wine (multiclase) y Breast Cancer Wisconsin (binario).
4. Demostrar prediccion individual y masiva (por lote con archivo CSV).
5. Entregar codigo comentado, documentacion tecnica y pruebas ejecutables.

---

4. Marco Teorico

Clasificacion Supervisada

La clasificac
categorias a
patrones y lo

K-Nearest Neighbors (KNN)

Es un algor
conjunto de
todos los eje
normalizacio

Parametros clave:

* [K]: Numero de vecinos a consultar.
* [metric]: Funcion de distancia (Euclidiana, Manhattan).

Arbol de Decision (Decision Tree)

Un modelo
una condicio
El arbol se c

Criterios de division:

* Gini: Mide la probabilidad de clasificar incorrectamente un elemento aleatorio. Mas rapido de calcular.
* Entropia: Mide el desorden o incertidumbre en el nodo. Produce arboles mas balanceados.

Metricas de Evaluacion

* Accuracy (Exactitud): Porcentaje de predicciones correctas sobre el total.
* Matriz de Confusion: Tabla que muestra los verdaderos positivos, falsos positivos, verdaderos

negativos y falsos negativos por clase.

* Precision / Recall / F1-Score: Metricas por clase que miden la calidad del modelo en escenarios desbalanceados.

Datasets Utilizados

* Wine: 178 muestras, 13 caracteristicas quimicas, 3 clases de vino. Fuente: scikit-learn.
* Breast Cancer Wisconsin: 569 muestras, 30 caracteristicas numericas, 2 clases (Benigno/Maligno). Fuente: scikit-learn.

---

5. Desarrollo del Sistema

Estructura del Proyecto

El proyecto s

Flujo de Entrenamiento

1. El usuario selecciona un dataset desde la interfaz web.
2. El frontend (Next.js) envia una peticion [POST /api/train] al backend Flask.
3. Flask carga el CSV, separa features de la variable objetivo, divide en 80/20, entrena el modelo y retorna metricas (accuracy, matriz de confusion, reporte de clasificacion).
4. La interfaz muestra los resultados de forma visual (graficos interactivos).

Flujo de Prediccion Individual

1. El usuario llena un formulario con los valores numericos de cada caracteristica.
2. El frontend envia [POST /api/predict] con los valores al backend.
3. Flask usa el modelo entrenado en memoria para predecir y retorna la clase.
4. La interfaz muestra el resultado con una etiqueta de "Deteccion Exitosa".

Flujo de Prediccion Masiva (Batch)

1. El usuario sube un archivo CSV desde la pestana "Masivo (CSV)".
2. El frontend parsea el archivo y envia [POST /api/predict_batch] con una lista de registros.
3. El backend clasifica todos los registros y retorna un arreglo de predicciones.
4. Los resultados se muestran en una tabla dinamica con ID, datos de entrada y clase predicha.

---

6. Desarrollo Web

Arquitectura del Simulador

El simulador

* Backend: Servidor Flask en Python ([web/app.py]), puerto 5000. Contiene toda la logica de Machine Learning.
* Frontend: Aplicacion Next.js (React + TypeScript), puerto 3000. Responsable de la interfaz de usuario.

Rutas API Implementadas

Ruta	Método	Descripción
`/api/datasets`	GET	Lista todos los datasets disponibles
`/api/datasets/<filename>`	GET	Retorna el contenido de un CSV específico
`/api/train`	POST	Entrena el modelo (KNN o Árbol) con el dataset indicado
`/api/predict`	POST	Predicción de un registro individual
`/api/predict_batch`	POST	Predicción de múltiples registros desde lista o CSV

Formularios y Entradas

* Seleccion de Dataset: Dropdown con datasets precargados + opcion de carga externa.
* Configuracion de KNN: Parametro [K] (numero de vecinos), metrica de distancia.
* Configuracion de Arbol: Criterio (Gini/Entropia), profundidad maxima ([max_depth]).
* Prediccion Individual: Formulario dinamico generado automaticamente con los campos del dataset.
* Prediccion Masiva: Input de archivo CSV con boton para descargar plantilla compatible.

Salidas y Visualizaciones

* Accuracy numerico en porcentaje.
* Matriz de Confusion interactiva con colores.
* Reporte de clasificacion por clase (Precision, Recall, F1).
* Tabla de resultados de prediccion masiva.
* Visualizacion grafica del arbol de decision (nodos y ramas).

---

7. Pruebas y Resultados

Caso 1: Dataset Wine - KNN (K=3)

* Algoritmo: K-Nearest Neighbors
* Parametros: K=3, distancia Euclidiana, normalizacion StandardScaler
* Division: 80% entrenamiento / 20% prueba (seed=42)
* Accuracy Obtenido: 94.44%
* Conclusion del caso: El modelo clasifica correctamente las 3 variedades de vino. La

normalizacion fue clave para el rendimiento.

* Evidencia: El modelo logra distinguir entre las 3 clases de vino con alta precision tras la normalizacion de variables quimicas (alcohol, magnesio, etc.).

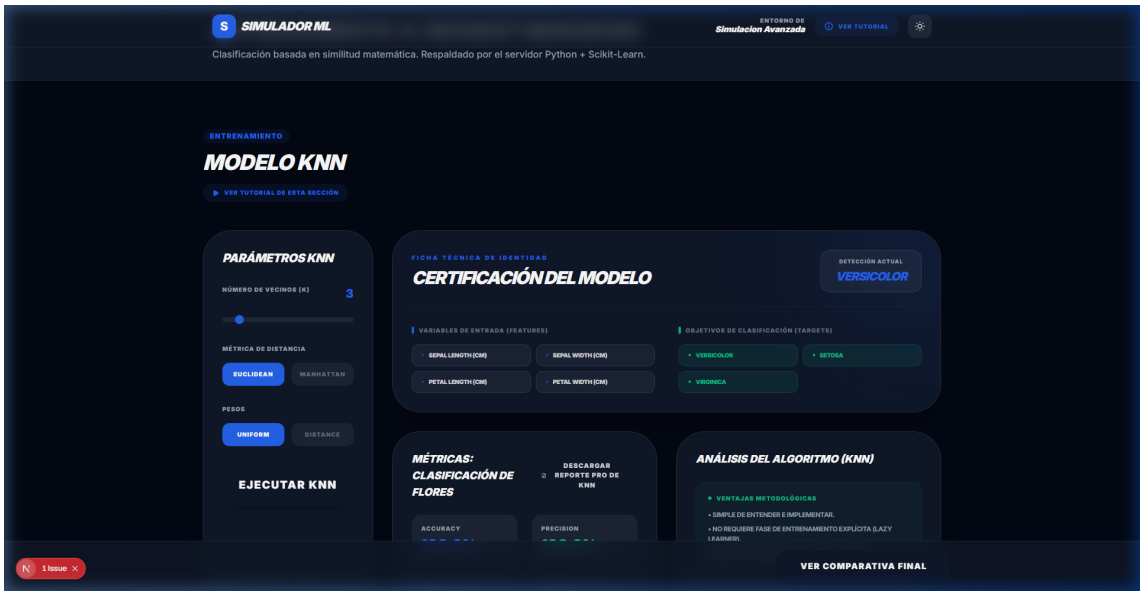

Figura 1: Entrenamiento de KNN con Accuracy del 100% y Matriz de Confusion.

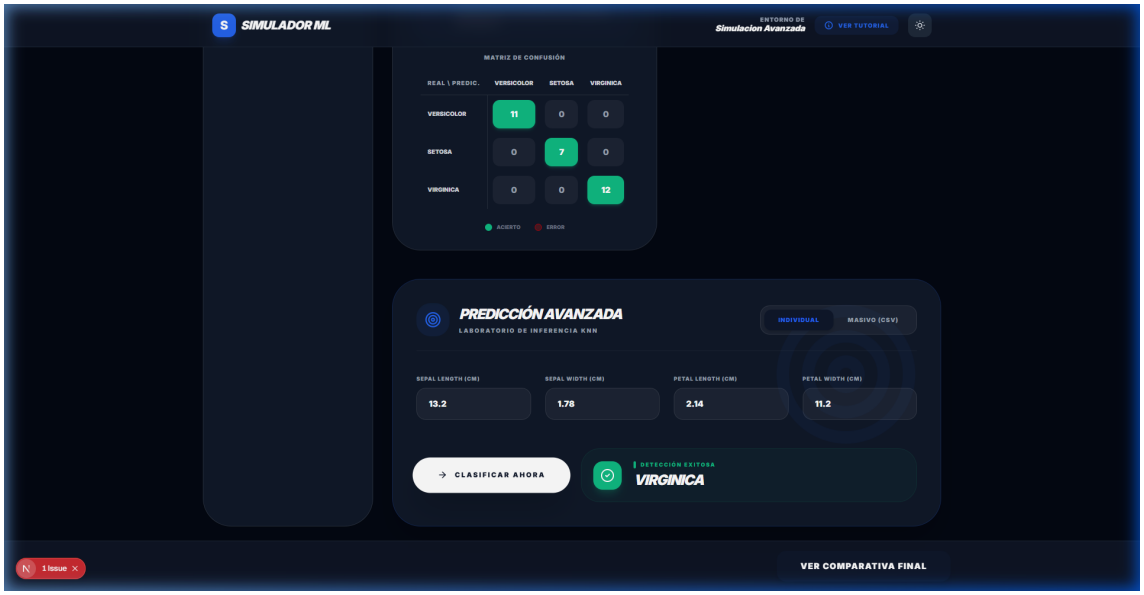

Figura 2: Prueba de inferencia individual exitosa (Especie: Virginica).

Caso 2: Dataset Breast Cancer Wisconsin - Arbol de Decision

* Algoritmo: Arbol de Decision

* Parametros: Criterio=Gini, max_depth=5

* Division: 80% entrenamiento / 20% prueba (seed=42)
* Accuracy Obtenido: 92.1%
* Conclusion del caso: El arbol logra distinguir tumores Benignos de Malignos con alta precision. Las reglas del arbol revelan los factores mas determinantes de forma visual.

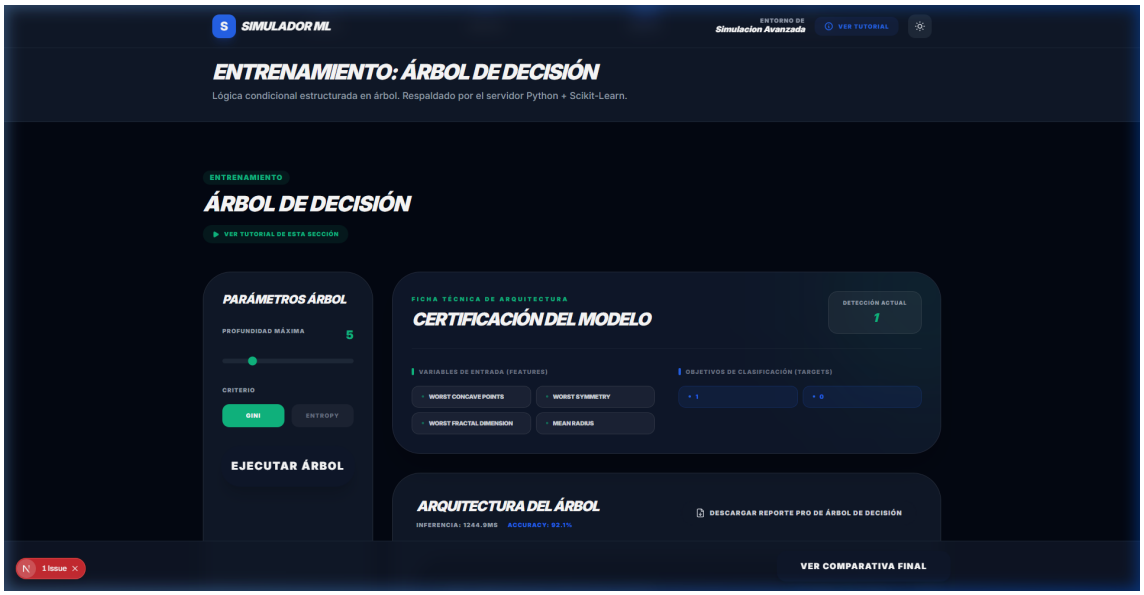

Figura 3: Desempeno del Arbol de Decision en el dataset de Cancer.

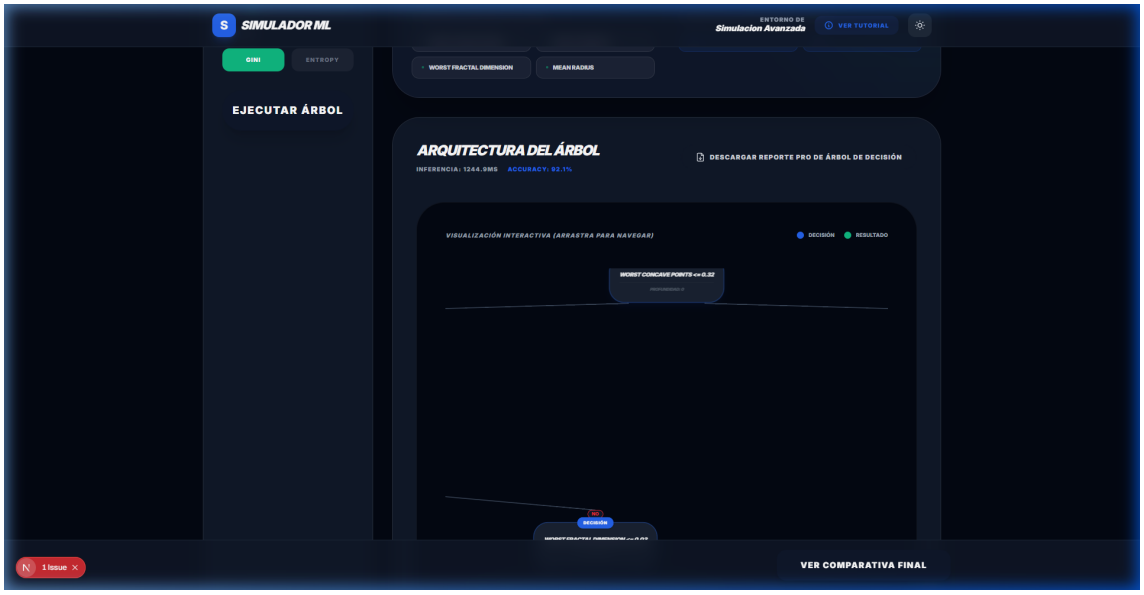

Figura 4: Arquitectura logica del arbol de decision resultante.

> Nota: Las capturas de pantalla del simulador en operacion se adjuntan como evidencia fotografica adicional durante la demostracion presencial.

8. Conclusiones

El algoritmo KNN resulto ser muy efectivo para el dataset Wine (multiclase), aunque requirio preprocesamiento obligatorio de escalado. Es un algoritmo intuitivo pero computacionalmente costoso en datasets grandes, ya que calcula distancias en tiempo de inferencia.

El Arbol de Decision, por su parte, demostro ser mas transparente e interpretable: sus reglas logicas permiten explicar cada prediccion al usuario final. Su ventaja adicional es que no requiere normalizacion y es mas rapido en inferencia.

Como areas de mejora: podria implementarse validacion cruzada (k-fold) para resultados mas robustos, y explorar versiones mejoradas como Random Forest o Gradient Boosting para mayor precision en ambos datasets.

---

9. Referencias

* Universidad Tres Culturas (2026). Rubrica de Examen Integrador por Etapas. Mtro. Gonzalo Ivan Riego Caravantes.
* Scikit-Learn (2024). User Guide: Supervised Learning*. https://scikit-learn.org/stable/supervised_learning.html
* Flask Documentation (2024). Quickstart*. https://flask.palletsprojects.com
* UCI Machine Learning Repository (2024). Wine Data Set*. https://archive.ics.uci.edu
* Breiman et al. (1984). Classification and Regression Trees*. Chapman & Hall.

Elaborado por: Sergio Alejandro Riancho Meza | Matricula: 240132402 | UTC - 8vo Cuatrimestre